

```

public int[] plusOne(int[] arr) {
    int n = arr.length;
    for(int i=arr.length-1;i>=0;i--){
        if(arr[i]<9){//199 -> 200
            arr[i]++;
            return arr;
        }else{
            arr[i] = 0;
        }
    }
    int ans[] = new int[n+1];
    ans[0] = 1;
    return ans;
}

```

Last Remaining After Alternate Max-Min Removal

Given an array **arr[]**, the size is reduced by 1 element at each step.

- In the first step, the maximum element would be removed.
- In the second step, the minimum element of the remaining array would be removed.
- In the third step again the maximum, and so on.
- Continue this till the array contains only one element.

Note:
Arrays.sort(arr);

Input: arr[] = [7, 8, 3, 4, 2, 9, 5]

Output: 5

Explanation: In first step '9' would be removed, in 2nd step '2' will be removed, in third step '8' will be removed and so on. So the last remaining element would be '5'.

Input: arr[] = [8, 1, 2, 9, 4, 3, 7, 5]

Output: 4

Explanation: In first step '9' would be removed, in 2nd step '1' will be removed, in third step '8' will be removed and so on. So the last remaining element would be '4'.

7	8	3	4	2	9	5
0	1	2	3	4	5	6
2	3	4	5	7	8	9
x	x	x		x	x	x

$$\text{len}/2 = 5$$

```

public int leftElement(int[] arr) {
    Arrays.sort(arr);
    int n = arr.length;
    if(n%2!=0){
        return arr[n/2];
    }
    return arr[(n/2)-1];
}

```

8	1	2	9	4	3	7	5
0	1	2	3	4	5	6	7
1	2	3	4	5	7	8	9
x	x	x		x	x	x	x

$$8/2 = 5 \times$$

$$\text{len}/2 - 1 = 4 \checkmark$$

If len is odd $\rightarrow n/2$.
 If len is even $\rightarrow n/2 - 1$.

Median of an Array

Input: arr[] = [90, 100, 78, 89, 67]

Output: 89

Explanation: After sorting the array middle element is the median

90	100	78	89	67
----	-----	----	----	----

67	78	89	90	100
----	----	----	----	-----

$n/2$

56	67	30	79
----	----	----	----

30	56	67	79
----	----	----	----

$(56 + 67) / 2.0$

Input: arr[] = [56, 67, 30, 79]

Output: 61.5

Explanation: In case of even number of elements, average of two middle elements is the median.

```
public double findMedian(int[] arr) {
    Arrays.sort(arr);
    int n = arr.length;
    if(n%2!=0){
        return arr[n/2];
    }
    return (arr[n/2] + arr[(n/2)-1])/2.0;
}
```

1464. Maximum Product of Two Elements in an Array

Input: nums = [3,4,5,2]

Output: 12

Explanation: If you choose the indices i=1 and j=2 (indexed from 0), you will get the maximum value, that is, (nums[1]-1)*(nums[2]-1) = (4-1)*(5-1) = 3*4 = 12.

```
class Solution {
    public int maxProduct(int[] nums) {
        Arrays.sort(nums);
        int n = nums.length;
        return (nums[(n - 1)] - 1) * (nums[(n - 2)] - 1);
    }
}
```

1491. Average Salary Excluding the Minimum and Maximum Salary

Input: salary = [4000,3000,1000,2000]

Output: 2500.00000

Explanation: Minimum salary and maximum salary are 1000 and 4000 respectively. Average salary excluding minimum and maximum salary is (2000+3000) / 2 = 2500

```
public double average(int[] salary) {
    Arrays.sort(salary);
    int n = salary.length;
    double sum = 0.0;
    for (int i = 1; i < n - 1; i++) {
        sum += salary[i];
    }
    return sum / (n - 2);
}
```

2455. Average Value of Even Numbers That Are Divisible by Three

Input: nums = [1,3,6,10,12,15]

Output: 9

Explanation: 6 and 12 are even numbers that are divisible by 3. $(6 + 12) / 2 = 9$.

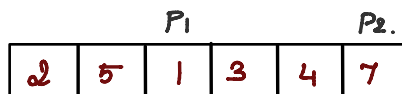
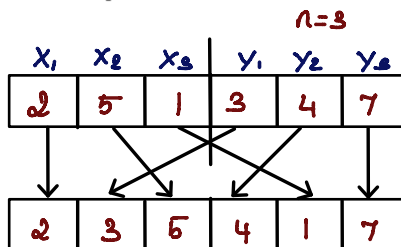
```
public int averageValue(int[] nums) {
    int cnt = 0, sum = 0;
    for(int i = 0; i < nums.length; i++){
        if(nums[i] % 2 == 0 && nums[i] % 3 == 0){
            sum += nums[i];
            cnt++;
        }
    }
    if(cnt == 0) return cnt;
    return sum / cnt;
}
```

1470. Shuffle the Array

Input: nums = [2,5,1,3,4,7], n = 3

Output: [2,3,5,4,1,7]

Explanation: Since $x_1=2$, $x_2=5$, $x_3=1$, $y_1=3$, $y_2=4$, $y_3=7$ then the answer is [2,3,5,4,1,7].



Ans =

2	3	5	4	1	7
---	---	---	---	---	---

- 1) Declare 2 pointers p_1 & p_2 .
 $p_1 = 0$ $p_2 = n$.
- 2) Create a answer array $\rightarrow$ size $2n$.
- 3) Write for loop from 0 to $n-1$.
- 4) In 1st iteration add p_1 & p_2 value in answer array.
- 5) update p_1 & p_2 .
- 6) Return answer.

```
public int[] shuffle(int[] nums, int n) {
    int p1 = 0, p2 = n;
    int ans[] = new int[n * 2];
    int idx = 0;
    for(int i = 0; i < n; i++){
        ans[idx++] = nums[p1++];
        ans[idx++] = nums[p2++];
    }
    return ans;
}
```

344. Reverse String

2 pointer technique

Input: s = ["h","e","l","l","o"]

Output: ["o","l","l","e","h"]

st end	
h	e
l	l
o	o

o	o
l	l
e	h

Declare 2 points st & end.

* st it starts from zero.

* end is starts from len-1.

Write while loop for iteration.

st < end

Swap the 2 points. 1st to last
& last to 1st

char temp = arr[st];

arr[st] = arr[end];

arr[end] = temp;

update st & end

st++; end--;

char temp = arr[st];

arr[st] = arr[end];

arr[end] = temp;

```
public void reverseString(char[] s) {
    int st = 0, end = s.length-1;
    while(st<end){
        char temp = s[st];
        s[st] = s[end];
        s[end] = temp;
        st++;
        end--;
    }
}
```

Reverse in Range.

ip:

7	8	3	4	2	9	5
---	---	---	---	---	---	---

st = 1 end = 4

op:

7	2	4	3	8	9	5
---	---	---	---	---	---	---

```
public static void ReverseInRange(int arr[], int st, int end){
    while(st<end){
        int temp = arr[st];
        arr[st] = arr[end];
        arr[end] = temp;
        st++;
        end--;
    }
}
```

189. Rotate Array

Input: nums = [1,2,3,4,5,6,7], k = 3

Output: [5,6,7,1,2,3,4]

Explanation:

rotate 1 steps to the right: [7,1,2,3,4,5,6]

rotate 2 steps to the right: [6,7,1,2,3,4,5]

rotate 3 steps to the right: [5,6,7,1,2,3,4]

1	2	3	4	5
---	---	---	---	---

k = 0, 5

5	1	2	3	4
---	---	---	---	---

k = 1, 6

4	5	1	2	3
---	---	---	---	---

k = 2, 7

3	4	5	1	2
---	---	---	---	---

k = 3, 8

2	3	4	5	1
---	---	---	---	---

k = 4, 9

1	2	3	4	5
---	---	---	---	---

k = 5 X

k → No of rotation.

		st		end
1	2	3	4	5

k = 3

n → len

5 - 3 = 2

n - k

st	end			
		n-k		n-1
1	2	5	4	3

0	n-k-1			
2	1	5	4	3

0				n-1
3	4	5	1	2

len = 5

5 - 3 = 2

ReverseInRange(arr, n-k, n-1);

ReverseInRange(arr, 0, n-k-1);

ReverseInRange(arr, 0, n-1);

```
public void ReverseInRange(int arr[], int st, int end){
    while(st<end){
        int temp = arr[st];
        arr[st] = arr[end];
        arr[end] = temp;
        st++;
        end--;
    }
}
```

```
public void rotate(int[] arr, int k) {
    int n = arr.length;
    k = k%n;
    ReverseInRange(arr, n-k, n-1);
    ReverseInRange(arr, 0, n-k-1);
    ReverseInRange(arr, 0, n-1);
}
```

k = 1003.

k = k % n; = 3

↓
To reduce the No of rotation.

283. Move Zeroes

Input: nums = [0,1,0,3,12]

Output: [1,3,12,0,0]

```
public void moveZeroes(int[] nums) {
    int n = nums.length;
    int j = 0;
    for (int i = 0; i < n; i++) {
        if (nums[i] != 0) {
            int temp = nums[i];
            nums[i] = nums[j];
            nums[j] = temp;
            j++;
        }
    }
}
```

1550. Three Consecutive Odds

Input: arr = [2,6,4,1]

Output: false

Explanation: There are no three consecutive odds.

```
public boolean threeConsecutiveOdds(int[] arr) {
    for(int i=0;i<arr.length-2;i++){
        if(arr[i]%2!=0 && arr[i+1]%2!=0 && arr[i+2]%2!=0){
            return true;
        }
    }
    return false;
}
```

1929. Concatenation of Array

Input: nums = [1,2,1]

Output: [1,2,1,1,2,1]

Explanation: The array ans is formed as follows:

- ans = [nums[0],nums[1],nums[2],nums[0],nums[1],nums[2]]

- ans = [1,2,1,1,2,1]

```
public int[] getConcatenation(int[] nums) {
    int n=nums.length;
    int ans[] = new int[n*2];
    for(int i=0;i<n;i++){
        ans[i]=nums[i];
        ans[i+n]=nums[i];
    }
    return ans;
}
```
